

AI-Powered Enterprise PPM

Intelligence & Transformation Platform

Complete Day-by-Day Build Guide — Modules 1 & 2

Layer	Technology	Purpose
Frontend	Next.js + TypeScript + Tailwind	UI & dashboards
Backend	FastAPI + PostgreSQL	APIs & data
AI Engine	LangChain + RAG + Agents	Intelligence layer
Analytics	Python ML Pipeline	Predictive models
DevOps	Docker + GitHub Actions	Deployment

■ What this guide covers

- **Week 1** — Environment setup (already completed ✓)
- **Module 1** — Enterprise Assessment Engine (Weeks 2-4, 12 days)
- **Module 2** — AI PMO Intelligence Assistant (Weeks 5-7, 12 days)
- Each day includes exact tasks, code snippets, file structure, and a completion checklist

Table of Contents

Week 1	Day 1	Environment Setup (Completed ✓)
Module 1	Days 2-4	Database Schema & Models
Module 1	Days 5-6	Assessment APIs & Business Logic
Module 1	Days 7-8	AI Semantic Evaluation Engine
Module 1	Days 9-10	Frontend — Assessment UI
Module 1	Days 11-13	Dashboard, Testing & Polish
Module 2	Days 14-16	Conversational AI Foundation
Module 2	Days 17-18	Multi-Agent Architecture
Module 2	Days 19-20	Analytics & Root Cause Engine
Module 2	Days 21-22	PMO Assistant UI
Module 2	Days 23-25	Integration, Testing & Launch

Week 1 — Day 1 ✓ Completed

- ✓ Congratulations! You have already completed Day 1. This section is a reference summary of what was built so you can verify your setup before moving to Module 1.

Completed Item	Verification Command / Check
VS Code + Extensions installed	Open VS Code → Extensions panel
Python 3.12+ installed	<code>python --version</code>
Node.js LTS installed	<code>node -v && npm -v</code>
Git initialized	<code>git log --oneline</code>
GitHub remote connected	<code>git remote -v</code>
PostgreSQL running	pgAdmin connects successfully
Virtual env created & activated	(venv) prefix in terminal
FastAPI running	<code>http://127.0.0.1:8000</code> returns JSON
Next.js running	<code>http://localhost:3000</code> loads
DB connected	<code>python app/database.py</code> → 'Database connected!'

- Before starting Module 1: make sure ALL verifications above pass. Fix any issues first — a broken foundation will slow down every day ahead.

MODULE 1 — Enterprise Assessment Engine

The Assessment Engine evaluates an organization's project management maturity across multiple dimensions — Agile, Governance, Delivery, and PMO capability. It uses AI to semantically interpret answers rather than simple checkbox scoring.

Feature	Description
Maturity Framework Builder	Create custom assessment frameworks per org
Dynamic Question Engine	Questions adapt based on previous answers
Department-Level Scoring	Score per team, BU, or department
AI Semantic Evaluation	LLM interprets free-text answers for maturity signals
PMO Capability Analysis	Structured scoring of PMO functions
Governance Evaluation	Assess governance maturity and compliance gaps
Delivery Scoring	Sprint velocity, success rates, on-time delivery

Module 1 Timeline Overview

Day	Focus
Day 2	Database schema — all assessment tables
Day 3	SQLAlchemy models + Alembic migrations
Day 4	Assessment seed data & admin panel setup
Day 5	Assessment CRUD APIs (create, read, submit)
Day 6	Scoring business logic & maturity calculation
Day 7	AI semantic evaluator — LangChain integration
Day 8	AI prompt engineering & evaluation pipeline
Day 9	Frontend — Assessment list & start page
Day 10	Frontend — Question flow & dynamic form
Day 11	Frontend — Results & maturity radar chart
Day 12	Full integration testing
Day 13	Polish, error handling, loading states, deploy

Goal

Design and create all database tables needed for the Assessment Engine. A well-thought-out schema now saves days of rework later.

Tables to Create

Table	Purpose
organizations	Stores company / client records
maturity_frameworks	Assessment framework definitions
assessment_categories	Sections within a framework (e.g. Agile, Governance)
assessment_questions	Individual questions per category
assessments	An assessment run for an org
assessment_responses	User answers to each question
maturity_scores	Computed scores per category + overall
ai_evaluations	AI interpretation of free-text answers

Step 1 — Write the SQL

Create file: **database/schema.sql**

```
-- Organizations

CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  industry VARCHAR(100),
  size VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);

-- Maturity Frameworks

CREATE TABLE maturity_frameworks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
```

```

);

-- Assessment Categories
CREATE TABLE assessment_categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  framework_id UUID REFERENCES maturity_frameworks(id),
  name VARCHAR(255) NOT NULL,
  weight DECIMAL(5,2) DEFAULT 1.0,
  order_index INTEGER DEFAULT 0
);

-- Questions
CREATE TABLE assessment_questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id UUID REFERENCES assessment_categories(id),
  question_text TEXT NOT NULL,
  question_type VARCHAR(50), -- scale/text/multi
  max_score INTEGER DEFAULT 5,
  order_index INTEGER DEFAULT 0
);

-- Assessment Run
CREATE TABLE assessments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id),
  framework_id UUID REFERENCES maturity_frameworks(id),
  status VARCHAR(50) DEFAULT 'in_progress',
  started_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP
);

-- Responses
CREATE TABLE assessment_responses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  assessment_id UUID REFERENCES assessments(id),
  question_id UUID REFERENCES assessment_questions(id),
  score INTEGER,

```

```
text_response TEXT,  
created_at TIMESTAMP DEFAULT NOW()  
);  
  
-- Computed Scores  
CREATE TABLE maturity_scores (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
assessment_id UUID REFERENCES assessments(id),  
category_id UUID REFERENCES assessment_categories(id),  
score DECIMAL(5,2),  
level VARCHAR(50), -- Initial/Developing/Defined/Managed/Optimizing  
ai_summary TEXT,  
computed_at TIMESTAMP DEFAULT NOW()  
);
```

Step 2 — Run Schema in pgAdmin

1. Open pgAdmin → your ppm_platform database
2. Click Query Tool
3. Paste the SQL above
4. Press F5 or click Execute
5. Verify all 8 tables appear in the left panel under Tables

Day 2 Checklist

- All 8 tables created in ppm_platform database
- UUID primary keys on all tables
- Foreign key relationships set up correctly
- Schema SQL committed to GitHub (git add . && git commit -m 'day2: assessment schema')

Goal

Create Python ORM models that mirror the schema, then set up Alembic so future schema changes are tracked and repeatable — a must for enterprise projects.

Step 1 — Create the Models

Create file: **backend/app/models/assessment.py**

```
from sqlalchemy import Column, String, Integer, Boolean, Text,
Numeric, DateTime, ForeignKey, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
import uuid
from app.database import Base

class Organization(Base):
    __tablename__ = 'organizations'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String(255), nullable=False)
    industry = Column(String(100))
    size = Column(String(50))
    created_at = Column(DateTime, server_default=func.now())
    assessments = relationship('Assessment', back_populates='organization')

class MaturityFramework(Base):
    __tablename__ = 'maturity_frameworks'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String(255), nullable=False)
    description = Column(Text)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, server_default=func.now())
    categories = relationship('AssessmentCategory', back_populates='framework')

class AssessmentCategory(Base):
    __tablename__ = 'assessment_categories'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```

framework_id = Column(UUID(as_uuid=True), ForeignKey('maturity_frameworks.id'))
name = Column(String(255), nullable=False)
weight = Column(Numeric(5, 2), default=1.0)
order_index = Column(Integer, default=0)
framework = relationship('MaturityFramework', back_populates='categories')
questions = relationship('AssessmentQuestion', back_populates='category')

# Add AssessmentQuestion, Assessment, AssessmentResponse,
# MaturityScore models following the same pattern

```

Step 2 — Set up Alembic

Inside the **backend/** folder with your venv activated:

```
alembic init alembic
```

Edit **alembic/env.py** — add these lines near the top:

```

from app.models import assessment # noqa – ensures models are registered
from app.database import Base
target_metadata = Base.metadata

```

Edit **alembic.ini** — set your database URL:

```
sqlalchemy.url = postgresql://postgres:YOUR_PASSWORD@localhost:5432/ppm_platform
```

Generate and apply the first migration:

```

alembic revision --autogenerate -m 'initial assessment tables'
alembic upgrade head

```

Day 3 Checklist

- All SQLAlchemy models created in backend/app/models/assessment.py
- Alembic initialized and env.py configured
- First migration generated (check alembic/versions/ for the file)
- alembic upgrade head runs without errors
- Committed to GitHub with message 'day3: sqlalchemy models + alembic'

Goal

Create a default maturity framework with categories and questions, and define Pydantic schemas so your APIs have clear request/response contracts.

Step 1 — Seed Script

Create file: **backend/app/seed.py**

```
from app.database import SessionLocal

from app.models.assessment import MaturityFramework, AssessmentCategory,
AssessmentQuestion

def seed():
    db = SessionLocal()
    # Create default framework
    fw = MaturityFramework(
        name='Enterprise PM Maturity Model',
        description='Assesses org-wide project management capability'
    )
    db.add(fw)
    db.flush()

    categories = [
        ('Agile Maturity', 1.2),
        ('Governance', 1.0),
        ('Delivery Capability', 1.1),
        ('PMO Functions', 1.0),
        ('Resource Management', 0.9),
    ]
    for i, (name, weight) in enumerate(categories):
        cat = AssessmentCategory(framework_id=fw.id, name=name,
                                weight=weight, order_index=i)
        db.add(cat)
        db.flush()

    # Add 5 questions per category
    for j in range(5):
```

```

q = AssessmentQuestion(
    category_id=cat.id,
    question_text=f'Describe your {name} practice #{j+1}',
    question_type='text',
    max_score=5,
    order_index=j
)

db.add(q)
db.commit()

print('Seed complete!')

if __name__ == '__main__':
    seed()

```

```
python app/seed.py
```

Step 2 — Pydantic Schemas

Create file: **backend/app/schemas/assessment.py**

```

from pydantic import BaseModel
from typing import Optional, List
from uuid import UUID
from datetime import datetime

class OrganizationCreate(BaseModel):
    name: str
    industry: Optional[str]
    size: Optional[str]

class OrganizationOut(OrganizationCreate):
    id: UUID
    created_at: datetime

class Config: from_attributes = True

class AssessmentCreate(BaseModel):
    org_id: UUID
    framework_id: UUID

```

```
class ResponseSubmit(BaseModel):  
    question_id: UUID  
    score: Optional[int]  
    text_response: Optional[str]  
  
class AssessmentSubmit(BaseModel):  
    responses: List[ResponseSubmit]
```

Day 4 Checklist

- Seed script runs and populates framework, categories, questions
- Verified data in pgAdmin — check all tables have rows
- Pydantic schemas created for Organization, Assessment, and Response
- Committed: 'day4: seed data + pydantic schemas'

Goal

Build the REST API endpoints that the frontend will call. By end of day, you should be able to start an assessment and submit responses via Postman.

Step 1 — Create the Router

Create file: **backend/app/routers/assessments.py**

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.assessment import (Organization, Assessment,
    AssessmentResponse, MaturityFramework, AssessmentQuestion)
from app.schemas.assessment import (OrganizationCreate, OrganizationOut,
    AssessmentCreate, AssessmentSubmit)
from typing import List

router = APIRouter(prefix='/api/assessments', tags=['assessments'])

# --- Organizations ---
@router.post('/organizations', response_model=OrganizationOut)
def create_org(payload: OrganizationCreate, db: Session = Depends(get_db)):
    org = Organization(**payload.dict())
    db.add(org)
    db.commit()
    db.refresh(org)
    return org

@router.get('/organizations', response_model=List[OrganizationOut])
def list_orgs(db: Session = Depends(get_db)):
    return db.query(Organization).all()

# --- Start Assessment ---
@router.post('/start')
def start_assessment(payload: AssessmentCreate, db: Session = Depends(get_db)):
    assessment = Assessment(**payload.dict())
```

```

db.add(assessment)

db.commit()

db.refresh(assessment)

# Fetch questions for this framework
questions = db.query(AssessmentQuestion).join(...).all()

return {'assessment_id': str(assessment.id), 'questions': questions}


# --- Submit Responses ---
@router.post('/{assessment_id}/submit')

def submit_responses(assessment_id: str, payload: AssessmentSubmit,
db: Session = Depends(get_db)):
    for r in payload.responses:
        resp = AssessmentResponse(
            assessment_id=assessment_id,
            question_id=str(r.question_id),
            score=r.score,
            text_response=r.text_response
        )
        db.add(resp)
        db.commit()

    return {'status': 'submitted', 'assessment_id': assessment_id}

```

Step 2 — Register Router in main.py

```

from app.routers import assessments

app.include_router(assessments.router)

```

Step 3 — Test in Postman

P /api/assessments/organizations — create an org, save the id
O
S
T

G /api/assessments/organizations — should list the org
E
T

P /api/assessments/start — body: {org_id, framework_id}
O
S
T

P /api/assessments/{id}/submit — body: {responses: [...]}
O
S
T

Day 5 Checklist

- All 4 endpoints return correct responses in Postman
- No 422 or 500 errors
- Swagger docs at /docs shows all routes
- Committed: 'day5: assessment CRUD APIs'

Goal

Implement the scoring engine that calculates a maturity score per category and an overall score, then maps it to a maturity level label.

Maturity Levels

Score Range	Maturity Level
0 – 1.0	Initial — Ad-hoc, unpredictable
1.1 – 2.0	Developing — Some processes defined
2.1 – 3.0	Defined — Standardized processes
3.1 – 4.0	Managed — Measured and controlled
4.1 – 5.0	Optimizing — Continuous improvement

Step 1 — Scoring Service

Create file: **backend/app/services/scoring.py**

```
from sqlalchemy.orm import Session

from app.models.assessment import (Assessment, AssessmentResponse,
AssessmentQuestion, AssessmentCategory, MaturityScore)

from decimal import Decimal

LEVELS = [
    (0.0, 1.0, 'Initial'),
    (1.1, 2.0, 'Developing'),
    (2.1, 3.0, 'Defined'),
    (3.1, 4.0, 'Managed'),
    (4.1, 5.0, 'Optimizing'),
]

def get_level(score: float) -> str:
    for low, high, label in LEVELS:
        if low <= score <= high:
            return label
    return 'Initial'
```

```

def compute_scores(assessment_id: str, db: Session) -> dict:
    responses = db.query(AssessmentResponse).filter_by(
        assessment_id=assessment_id).all()
    # Group responses by category
    cat_responses: dict = {}
    for r in responses:
        q = db.query(AssessmentQuestion).get(r.question_id)
        cat_id = str(q.category_id)
        cat_responses.setdefault(cat_id, []).append(r)
    # Compute per-category score
    results = {}
    for cat_id, resps in cat_responses.items():
        cat = db.query(AssessmentCategory).get(cat_id)
        scored = [r for r in resps if r.score is not None]
        avg = sum(r.score for r in scored) / len(scored) if scored else 0
        score_obj = MaturityScore(
            assessment_id=assessment_id,
            category_id=cat_id,
            score=round(avg, 2),
            level=get_level(avg)
        )
        db.add(score_obj)
        results[cat.name] = {'score': avg, 'level': get_level(avg)}
    db.commit()
    return results

```

Step 2 — Trigger Scoring on Submit

In your submit endpoint, after saving responses, call:

```

from app.services.scoring import compute_scores

# After db.commit() in submit_responses:
scores = compute_scores(assessment_id, db)
return {'status': 'complete', 'scores': scores}

```

Day 6 Checklist

- Scoring service correctly averages scores per category
- Maturity level labels are assigned based on score ranges

-
- MaturityScore records saved to database after submission
 - Tested end-to-end in Postman — submit responses → get scores back
 - Committed: 'day6: scoring business logic'

DAY
7

AI Semantic Evaluator — LangChain Setup

Integrate OpenAI/Gemini to interpret free-text answers

Goal — Days 7 & 8

This is the key AI differentiator of Module 1. Instead of only scoring numeric answers, the AI reads free-text responses and identifies maturity patterns, gaps, and improvement suggestions.

Step 1 — Install AI Dependencies

```
pip install langchain langchain-openai openai python-dotenv  
pip freeze > requirements.txt
```

Add to your `.env` file:

```
OPENAI_API_KEY=sk-your-key-here
```

■ Never commit your `.env` file to GitHub. Verify `.gitignore` includes `.env` before committing.

Step 2 — AI Evaluation Service

Create file: `backend/app/services/ai_evaluator.py`

```
from langchain_openai import ChatOpenAI  
from langchain.prompts import PromptTemplate  
from langchain.chains import LLMChain  
import os  
  
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.2,  
openai_api_key=os.getenv('OPENAI_API_KEY'))  
  
EVAL_PROMPT = PromptTemplate(  
input_variables=['category', 'question', 'answer'],  
template='''  
You are an enterprise project management maturity expert.  
Category: {category}  
Question: {question}  
User Answer: {answer}  
  
Evaluate this answer and respond ONLY with a valid JSON object:  
{  
  "score": <integer 1-5>,  
  "level": "<Initial|Developing|Defined|Managed|Optimizing>,"
```

```

    "gaps": ["gap1", "gap2"],
    "strengths": ["strength1"],
    "recommendation": "<one actionable improvement>"
  }}
  '''
)

chain = LLMChain(llm=llm, prompt=EVAL_PROMPT)

def evaluate_response(category: str, question: str, answer: str) -> dict:
    result = chain.invoke({
        'category': category,
        'question': question,
        'answer': answer
    })
    import json
    return json.loads(result['text'])

```

Step 3 — Wire AI into the Submit Flow (Day 8)

In your submit endpoint, for text responses, call the AI evaluator:

```

from app.services.ai_evaluator import evaluate_response

for r in payload.responses:
    if r.text_response: # Only evaluate text answers
        q = db.query(AssessmentQuestion).get(str(r.question_id))
        cat = db.query(AssessmentCategory).get(str(q.category_id))
        ai_result = evaluate_response(
            category=cat.name,
            question=q.question_text,
            answer=r.text_response
        )
        r.score = ai_result.get('score', 3) # Use AI score
        # Store full AI evaluation in ai_evaluations table

```

Days 7-8 Checklist

- LangChain and OpenAI packages installed
- OPENAI_API_KEY in .env (not committed to git)

-
- AI evaluator returns valid JSON with score, level, gaps, recommendation
-
- Text responses now get AI-scored automatically
-
- Tested with a real free-text answer — verify AI output makes sense
-
- Committed: 'day7-8: AI semantic evaluator'

Goal — Days 9 & 10

Build the frontend pages a user will interact with: browsing available frameworks, starting an assessment, answering questions, and seeing results.

Pages to Build

Page / Component	Route
Assessment List	/assessments
Start Assessment Modal	component in /assessments
Question Flow	/assessments/[id]/questions
Results Dashboard	/assessments/[id]/results

Step 1 — API Service Layer

Create file: **frontend/src/lib/api.ts**

```
import axios from 'axios'

const api = axios.create({ baseURL: 'http://localhost:8000' })

export const startAssessment = (orgId: string, frameworkId: string) =>
  api.post('/api/assessments/start', { org_id: orgId, framework_id: frameworkId })

export const submitResponses = (assessmentId: string, responses: any[]) =>
  api.post(`/api/assessments/${assessmentId}/submit`, { responses })

export const getResults = (assessmentId: string) =>
  api.get(`/api/assessments/${assessmentId}/results`)
```

Step 2 — Question Form Component (Day 10)

Create file: **frontend/src/components/QuestionCard.tsx**

```
interface QuestionCardProps {
  question: { id: string; question_text: string; question_type: string }
  onAnswer: (questionId: string, score: number, text: string) => void
}

export function QuestionCard({ question, onAnswer }: QuestionCardProps) {
```

```

const [score, setScore] = useState(3)
const [text, setText] = useState('')

return (
  <div className='border rounded-lg p-6 shadow-sm bg-white'>
    <p className='text-lg font-medium mb-4'>{question.question_text}</p>
    { /* Slider for numeric score */ }
    <input type='range' min={1} max={5} value={score}
      onChange={e => setScore(+e.target.value)} />
    { /* Textarea for free text */ }
    <textarea className='w-full border rounded p-2 mt-3'
      placeholder='Describe your current practice...'
      value={text} onChange={e => setText(e.target.value)} />
    <button className='mt-4 bg-blue-600 text-white px-4 py-2 rounded'
      onClick={() => onAnswer(question.id, score, text)}>
      Next Question
    </button>
  </div>
)
}

```

Days 9-10 Checklist

- Assessment list page shows available frameworks
- User can start a new assessment (creates record in DB)
- Question flow navigates through all questions
- Each question accepts slider score AND free-text response
- Submission calls the API and redirects to results
- No CORS errors (add FastAPI CORSMiddleware if needed)
- Committed: 'day9-10: assessment frontend'

Goal

Display assessment results as a maturity radar chart plus a breakdown table with AI-generated recommendations per category.

Step 1 — Radar Chart Component

```
import { RadarChart, Radar, PolarGrid,
PolarAngleAxis, Tooltip, ResponsiveContainer } from 'recharts'

export function MaturityRadar({ scores }: { scores: Record<string, any> }) {
  const data = Object.entries(scores).map(([name, val]) => ({
    subject: name,
    score: val.score,
    fullMark: 5,
  }))

  return (
    <ResponsiveContainer width='100%' height={350}>
      <RadarChart data={data}>
        <PolarGrid />
        <PolarAngleAxis dataKey='subject' />
        <Radar dataKey='score' fill='#2563EB' fillOpacity={0.5}
stroke='#2563EB' />
        <Tooltip />
      </RadarChart>
    </ResponsiveContainer>
  )
}
```

Step 2 — Results Page Layout

The results page should show:

- Radar chart of all category scores
- Overall maturity level badge (e.g. 'Defined')
- Per-category table: score, level, AI recommendation

-
- Top 3 improvement priorities highlighted
 - Export to PDF button (can be a simple `print:window.print()`)
 -
-

Day 11 Checklist

- Radar chart renders with real scores from the API
 - Overall maturity level displayed prominently
 - Per-category breakdown table shows AI recommendations
 - Page looks professional (shadcn Card components, clean layout)
 - Committed: 'day11: results dashboard'
-

DAY
12

Testing, Polish & Module 1 Wrap-up

End-to-end testing, error handling, and deployment prep

Goal — Days 12 & 13

Do a complete end-to-end run of Module 1, fix edge cases, add loading states, and commit a clean tagged release.

Testing Scenarios

- Create a new organization
- Start an assessment with that org
- Answer all questions (mix of numeric + text)
- Submit and verify AI scoring runs without error
- View results — radar chart renders, scores are correct
- Try submitting with some questions unanswered — check error handling
- Try with a very long text response — AI still returns valid JSON
- Refresh results page — data loads correctly from DB

Polish Checklist

- Loading spinners shown while API calls are in progress
- Error messages shown to user for failed API calls (not just console.error)
- Empty states handled (no assessments yet, no org yet)
- Mobile responsive — test at 375px width
- FastAPI error responses use proper HTTP status codes (404, 422, 500)
- AI evaluation errors don't crash the whole submit — graceful fallback to numeric score

Module 1 Release

```
git add .  
git commit -m 'feat: Module 1 - Enterprise Assessment Engine complete'  
git tag v1.0-module1  
git push origin main --tags
```

- Module 1 is now complete! You have a working AI-powered maturity assessment system. Recruiters and reviewers can see: PostgreSQL schema design, FastAPI REST APIs, SQLAlchemy ORM, LangChain AI integration, and a Next.js frontend with data visualization.

MODULE 2 — AI PMO Intelligence Assistant

Module 2 is your major innovation layer. It adds a conversational AI assistant that analyzes project data, answers natural-language questions, performs root cause analysis on delays, and generates actionable PMO recommendations.

Capability	How It Works
Conversational AI	User asks questions in plain English, AI answers with project context
Root Cause Analysis	AI analyzes sprint velocity, risks, resource data to find delays
Multi-Agent Architecture	Specialized agents for risk, resources, timeline analysis
Natural Language Analytics	AI converts 'show delayed projects' to DB queries
AI Recommendations	Structured action plans saved to DB and shown on dashboard

Module 2 Timeline Overview

Day	Focus
Day 14	New DB tables — projects, conversations, AI messages
Day 15	Project CRUD APIs + dummy project data
Day 16	LangChain conversational chain with project context
Day 17	Multi-agent setup — Risk Agent + Resource Agent
Day 18	Timeline Analysis Agent + Agent Orchestrator
Day 19	Root cause analysis pipeline
Day 20	Natural language to SQL converter
Day 21	Frontend — PMO Assistant chat UI
Day 22	Frontend — AI recommendations panel
Day 23	Integration testing across agents
Day 24	Performance optimization + streaming responses
Day 25	Module 2 release + full project demo prep

New Database Tables for Module 2

Projects, conversations, AI messages, recommendations

Goal

Add all database tables needed to support project tracking and AI conversation history.

New Tables

Table	Purpose
projects	Project records (name, status, budget, timeline)
project_risks	Identified risks per project
project_resources	Team members and allocation
sprint_data	Sprint velocity and completion data
conversations	AI chat session per user
ai_messages	Individual messages in each conversation
ai_recommendations	Structured recommendations saved by AI agents

Key Schema for Projects

```
CREATE TABLE projects (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  org_id UUID REFERENCES organizations(id),  
  name VARCHAR(255) NOT NULL,  
  status VARCHAR(50), -- on_track / at_risk / delayed / completed  
  health_score DECIMAL(5,2),  
  budget DECIMAL(15,2),  
  budget_used DECIMAL(15,2),  
  start_date DATE,  
  target_end_date DATE,  
  actual_end_date DATE,  
  team_size INTEGER,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE conversations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  org_id UUID REFERENCES organizations(id),  
  title VARCHAR(255),
```

```

created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE ai_messages (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
conversation_id UUID REFERENCES conversations(id),
role VARCHAR(20), -- user / assistant
content TEXT NOT NULL,
agent_used VARCHAR(100),
created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE ai_recommendations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
org_id UUID REFERENCES organizations(id),
project_id UUID REFERENCES projects(id),
type VARCHAR(100), -- risk / resource / timeline
priority VARCHAR(20), -- high / medium / low
title VARCHAR(255),
description TEXT,
action_items JSONB,
created_at TIMESTAMP DEFAULT NOW()
);

```

Day 14 Checklist

- All 7 new tables created in pgAdmin
- Alembic migration generated and applied
- SQLAlchemy models created for all new tables
- Committed: 'day14: Module 2 database schema'

Goal

Build project management APIs and seed realistic project data so the AI assistant has meaningful data to analyze.

Step 1 — Project APIs

```
router = APIRouter(prefix='/api/projects', tags=['projects'])

@router.get('/')
def list_projects(org_id: Optional[str] = None, db: Session = Depends(get_db)):
    q = db.query(Project)
    if org_id:
        q = q.filter_by(org_id=org_id)
    return q.all()

@router.get('/{project_id}/summary')
def project_summary(project_id: str, db: Session = Depends(get_db)):
    project = db.query(Project).get(project_id)
    risks = db.query(ProjectRisk).filter_by(project_id=project_id).all()
    sprints = db.query(SprintData).filter_by(project_id=project_id).all()
    return {
        'project': project,
        'risks': risks,
        'sprints': sprints,
        'health_score': project.health_score
    }
```

Step 2 — Realistic Seed Data

Add to your seed.py — create 5 sample projects with varying health:

```
projects = [
    {'name': 'ERP Modernization', 'status': 'at_risk',
     'health_score': 62, 'budget': 500000, 'budget_used': 340000,
     'team_size': 12},
    {'name': 'Mobile App Launch', 'status': 'on_track',
     'health_score': 88, 'budget': 150000, 'budget_used': 95000,
```

```
'team_size': 6},  
{ 'name': 'Data Migration', 'status': 'delayed',  
  'health_score': 41, 'budget': 200000, 'budget_used': 190000,  
  'team_size': 4},  
# ... add 2 more  
]
```

Day 15 Checklist

- Project list and detail APIs working
- 5 sample projects seeded with risks and sprint data
- GET /api/projects returns correct list
- GET /api/projects/{id}/summary returns full context
- Committed: 'day15: project APIs + seed data'

Goal

Build the core AI chat capability: user sends a question, the AI receives project context and conversation history, and returns an intelligent answer.

Step 1 — Context Builder

Create file: **backend/app/services/context_builder.py**

```
def build_project_context(org_id: str, db) -> str:
    projects = db.query(Project).filter_by(org_id=org_id).all()
    context_parts = []
    for p in projects:
        risks = db.query(ProjectRisk).filter_by(project_id=str(p.id)).all()
        risk_summary = ', '.join([r.description for r in risks[:3]])
        context_parts.append(
            f'Project: {p.name} | Status: {p.status} | '
            f'Health: {p.health_score}/100 | '
            f'Budget used: {p.budget_used}/{p.budget} | '
            f'Team: {p.team_size} | Top risks: {risk_summary}'
        )
    return '\n'.join(context_parts)
```

Step 2 — Conversational Chain

Create file: **backend/app/services/pmo_assistant.py**

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.schema import HumanMessage, AIMessage

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.3)

SYSTEM_PROMPT = '''
You are an expert AI PMO Intelligence Assistant.
You have access to the following project portfolio data:
{project_context}

Answer questions about project health, risks, delays, and resource usage.
```

```

Always be specific, reference actual project names and numbers.
If recommending actions, be concrete and prioritized.
'''

def chat(question: str, history: list, project_context: str) -> str:
    messages = []
    for msg in history:
        if msg['role'] == 'user':
            messages.append(HumanMessage(content=msg['content']))
        else:
            messages.append(AIMessage(content=msg['content']))
    messages.append(HumanMessage(content=question))

    prompt = ChatPromptTemplate.from_messages([
        ('system', SYSTEM_PROMPT),
        MessagesPlaceholder(variable_name='history'),
        ('human', '{question}'),
    ])
    chain = prompt | llm
    response = chain.invoke({
        'project_context': project_context,
        'history': messages[:-1],
        'question': question
    })
    return response.content

```

Step 3 — Chat API Endpoint

```

@router.post('/api/assistant/chat')
def chat_endpoint(payload: ChatRequest, db: Session = Depends(get_db)):
    context = build_project_context(payload.org_id, db)
    # Load conversation history from DB
    history = load_history(payload.conversation_id, db)
    response = chat(payload.message, history, context)
    # Save both messages to DB
    save_message(payload.conversation_id, 'user', payload.message, db)
    save_message(payload.conversation_id, 'assistant', response, db)
    return {'response': response}

```

Day 16 Checklist

- Context builder returns meaningful project summary string
- Chat endpoint accepts message + conversation_id + org_id
- AI responds with project-specific insights (mentions actual project names)
- Conversation history persisted to ai_messages table
- Tested question: 'Which projects are at highest risk?' — AI names them
- Committed: 'day16: conversational AI chain'

Goal — Days 17 & 18

Instead of one general AI, build specialized agents that are experts in their domain. An orchestrator routes questions to the right agent.

Agent Design

Agent	Responsibility
Risk Agent	Analyzes risk logs, scores project risk, identifies critical risks
Resource Agent	Checks team allocation, spots overloading and burnout signals
Timeline Agent	Analyzes sprint velocity, predicts delays, identifies bottlenecks
Orchestrator	Classifies the user's question and routes to the right agent

Step 1 — Base Agent Pattern

Create file: **backend/app/agents/base_agent.py**

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from abc import ABC, abstractmethod

class BaseAgent(ABC):
    def __init__(self, system_prompt: str):
        self.llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.2)
        self.system_prompt = system_prompt

    def run(self, context: str, question: str) -> dict:
        prompt = ChatPromptTemplate.from_messages([
            ('system', self.system_prompt),
            ('human', 'Context:\n{context}\n\nQuestion: {question}'),
        ])
        chain = prompt | self.llm
        result = chain.invoke({'context': context, 'question': question})
        return {'agent': self.__class__.__name__, 'response': result.content}
```

Step 2 — Risk Agent

Create file: **backend/app/agents/risk_agent.py**

```
from app.agents.base_agent import BaseAgent
```

```

RISK_PROMPT = '''
You are an expert project risk analyst with 15 years of enterprise experience.
Analyze the project data and answer risk-related questions.
Structure your response with:
1. Risk Assessment Summary
2. Critical Risks (Top 3)
3. Recommended Mitigations
4. Risk Score (1-10)
'''

class RiskAgent(BaseAgent):
    def __init__(self):
        super().__init__(RISK_PROMPT)

```

Step 3 — Orchestrator (Day 18)

Create file: **backend/app/agents/orchestrator.py**

```

from app.agents.risk_agent import RiskAgent
from app.agents.resource_agent import ResourceAgent
from app.agents.timeline_agent import TimelineAgent

AGENTS = {
    'risk': RiskAgent(),
    'resource': ResourceAgent(),
    'timeline': TimelineAgent(),
}

def classify_question(question: str) -> str:
    '''Simple keyword routing – upgrade to LLM classification later'''
    q = question.lower()
    if any(w in q for w in ['risk', 'issue', 'blocker', 'critical']):
        return 'risk'
    if any(w in q for w in ['resource', 'team', 'burnout', 'allocation']):
        return 'resource'
    if any(w in q for w in ['delay', 'timeline', 'deadline', 'sprint']):
        return 'timeline'
    return 'risk' # default

```

```
def route(question: str, context: str) -> dict:
    agent_key = classify_question(question)
    agent = AGENTS[agent_key]
    return agent.run(context, question)
```

Days 17-18 Checklist

- BaseAgent abstract class created
- RiskAgent, ResourceAgent, TimelineAgent all created with specialized prompts
- Orchestrator correctly routes questions based on keywords
- Each agent tested with a sample question — responses are domain-specific
- Chat endpoint updated to use orchestrator instead of direct LLM call
- Committed: 'day17-18: multi-agent architecture'

Goal — Days 19 & 20

Two power features: (1) When a user asks 'Why is this project delayed?', run a structured root-cause analysis pipeline. (2) When a user asks a data query in English, convert it to SQL and return results.

Root Cause Analysis Pipeline (Day 19)

Create file: **backend/app/services/root_cause.py**

```
def analyze_root_cause(project_id: str, db) -> dict:
    project = db.query(Project).get(project_id)
    risks = db.query(ProjectRisk).filter_by(project_id=project_id).all()
    sprints = db.query(SprintData).filter_by(project_id=project_id).all()

    # Build analysis context
    context = f'''
    Project: {project.name}
    Status: {project.status}
    Health Score: {project.health_score}/100
    Budget: {project.budget_used}/{project.budget} used
    Sprint Velocity Trend: {[s.velocity for s in sprints[-5:]]}
    Top Risks: {[r.description for r in risks]}
    '''

    prompt = f'''
    Analyze the following project data and identify the root causes of delays.
    {context}
    Respond with JSON:
    {{
    "primary_cause": "...",
    "contributing_factors": ["...", "..."],
    "impact_assessment": "...",
    "recommended_actions": ["action1", "action2", "action3"],
    "recovery_timeline": "..."
    }}
    '''

    result = llm.invoke(prompt)
```

```
return json.loads(result.content)
```

Natural Language to SQL (Day 20)

Create file: **backend/app/services/nl_to_sql.py**

```
NL_SQL_PROMPT = '''

You are a PostgreSQL expert. Convert the user's natural language question
into a safe SELECT SQL query against these tables:

- projects (id, name, status, health_score, budget, budget_used, team_size)
- project_risks (id, project_id, description, severity)
- sprint_data (id, project_id, sprint_number, velocity, completion_rate)

Rules:

- Only generate SELECT statements
- Always LIMIT to 50 rows
- Return ONLY the SQL, no explanation
'''

def nl_to_sql(question: str) -> str:
    prompt = f'{NL_SQL_PROMPT}\n\nQuestion: {question}'
    result = llm.invoke(prompt)
    return result.content.strip()

def execute_nl_query(question: str, db) -> dict:
    sql = nl_to_sql(question)
    # Safety check – only allow SELECT
    if not sql.upper().strip().startswith('SELECT'):
        return {'error': 'Invalid query generated'}
    result = db.execute(text(sql))
    rows = [dict(row) for row in result]
    return {'query': sql, 'results': rows}
```

Days 19-20 Checklist

- Root cause analysis returns structured JSON for a delayed project
- NL to SQL converts 'show all delayed projects' to correct SQL
- Safety check prevents non-SELECT queries from executing
- Both services exposed as API endpoints
- Tested with 5 different natural language questions — all return valid results

■ Committed: 'day19-20: root cause analysis + NL to SQL'

Goal — Days 21 & 22

Build the frontend for the PMO Assistant: a chat interface and a recommendations panel.

Chat UI Component

```
export function PmoAssistant() {
  const [messages, setMessages] = useState<Message[]>([])
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)

  const sendMessage = async () => {
    if (!input.trim()) return
    const userMsg = { role: 'user', content: input }
    setMessages(prev => [...prev, userMsg])
    setInput('')
    setLoading(true)

    const res = await fetch('/api/assistant/chat', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: input, org_id: currentOrgId,
        conversation_id: conversationId })
    })

    const data = await res.json()
    setMessages(prev => [...prev, { role: 'assistant', content: data.response }])
    setLoading(false)
  }

  return (
    <div className='flex flex-col h-screen max-w-3xl mx-auto'>
      <div className='flex-1 overflow-y-auto p-4 space-y-4'>
        {messages.map((m, i) => (
          <div key={i} className={m.role === 'user' ? 'text-right' : 'text-left'}>
            <div className={`inline-block p-3 rounded-lg max-w-xl
              ${m.role==='user' ? 'bg-blue-600 text-white' : 'bg-gray-100'}`}>
              {m.content}
            </div>
          </div>
        ))}
      </div>
    </div>
  )
}
```

```

</div>
</div>
)}}
{loading && <div>AI is thinking...</div>}
</div>
<div className='p-4 border-t flex gap-2'>
  <input className='flex-1 border rounded-lg px-4 py-2'
    value={input} onChange={e => setInput(e.target.value)}
    onKeyDown={e => e.key === 'Enter' && sendMessage()} />
  <button onClick={sendMessage}
    className='bg-blue-600 text-white px-6 py-2 rounded-lg'>
    Send
  </button>
</div>
</div>
)
}

```

Suggested Starter Questions Panel (Day 22)

Add clickable suggested questions to help users discover the AI's capabilities:

- "Which projects are at highest risk this quarter?"
- "Why is the ERP Modernization project delayed?"
- "Show me projects with budget overruns."
- "Which team has the highest workload?"
- "What should I prioritize this week?"

Days 21-22 Checklist

- Chat interface renders messages in a scrollable container
- User messages right-aligned, AI messages left-aligned (WhatsApp style)
- Loading indicator shown while AI responds
- Suggested starter questions are clickable and populate input
- Recommendations panel shows AI-generated action items
- Committed: 'day21-22: PMO assistant UI'

DAY
23

Integration, Streaming & Module 2 Launch

Polish, streaming responses, and final release

Goal — Days 23, 24 & 25

Final phase: add response streaming for a better UX, do full integration testing, and tag the Module 2 release.

Day 23 — Full Integration Testing

- Complete a full user journey: create org → run assessment → open PMO assistant
- Ask 10 different questions covering risk, resource, timeline domains
- Verify orchestrator routes correctly for each question type
- Test root cause analysis on the 'delayed' project
- Test NL to SQL with 5 different natural language queries
- Conversation history persists across page refresh

Day 24 — Streaming Responses

Add streaming so users see AI responses token by token (much better UX).

```
# FastAPI streaming endpoint
from fastapi.responses import StreamingResponse

@router.post('/api/assistant/chat/stream')
async def chat_stream(payload: ChatRequest, db: Session = Depends(get_db)):
    context = build_project_context(payload.org_id, db)
    history = load_history(payload.conversation_id, db)

    async def generate():
        async for chunk in llm.astream(build_messages(payload.message, history, context)):
            yield f'data: {chunk.content}\n\n'
        yield 'data: [DONE]\n\n'

    return StreamingResponse(generate(), media_type='text/event-stream')
```

Frontend — consume the stream:

```
const res = await fetch('/api/assistant/chat/stream', { method: 'POST', ... })
const reader = res.body!.getReader()

let assistantMsg = ''
while (true) {
    const { done, value } = await reader.read()
```

```
if (done) break

const chunk = new TextDecoder().decode(value)

assistantMsg += chunk.replace('data: ', '').replace('[DONE]', '')

setStreamingMessage(assistantMsg) // Update UI in real time

}
```

Day 25 — Module 2 Release

- All agent types (Risk, Resource, Timeline) working correctly
- Streaming responses working in the frontend
- Conversation history correctly maintained
- Error handling for OpenAI API failures (rate limits, timeouts)
- README.md updated with Module 2 feature documentation
- Demo script prepared: 3-minute walkthrough of the full system

```
git add .

git commit -m 'feat: Module 2 - AI PMO Intelligence Assistant complete'

git tag v2.0-module2

git push origin main --tags
```

- Congratulations! You now have a production-quality AI PMO Intelligence Platform with Module 1 (Assessment Engine) and Module 2 (AI Assistant) fully operational. This demonstrates: LangChain, multi-agent AI, conversational AI, NL to SQL, streaming responses, PostgreSQL, FastAPI, and a modern Next.js frontend.

Quick Reference

Daily Startup Commands

```
# 1. Activate Python venv (backend terminal)
cd backend && venv\Scripts\activate # Windows
cd backend && source venv/bin/activate # Mac/Linux

# 2. Start FastAPI
uvicorn app.main:app --reload

# 3. Start Next.js (separate terminal)
cd frontend && npm run dev

# 4. Verify PostgreSQL is running
# Open pgAdmin – should connect without error
```

Git Workflow

```
git add .
git commit -m 'dayX: description of what was built'
git push origin main
```

Useful URLs

Service	URL
FastAPI Backend	http://127.0.0.1:8000
Swagger API Docs	http://127.0.0.1:8000/docs
Next.js Frontend	http://localhost:3000
pgAdmin	http://localhost:5050 or installed app

Common Issues & Fixes

Problem	Fix
CORS error in browser	Add CORSMiddleware to FastAPI main.py
422 Unprocessable Entity	Check request body matches Pydantic schema
OpenAI API rate limit	Add time.sleep(1) between AI calls in batch mode
Alembic migration conflict	alembic downgrade -1 then re-generate migration
DB connection refused	Check PostgreSQL service is running in Task Manager

venv not activating	Delete venv/ folder and recreate it
---------------------	-------------------------------------